What is Docker? — Simple words-

Docker is a tool that packages an application and everything it needs to run into a container.

Why do we use Docker?

The common problems without Docker are:

Developer Machine
↓
"It works on my machine"
↓
Test Environment
↓
Dependency/Configuration issues
↓
Production Failure

Docker helps by packaging everything required by the application:

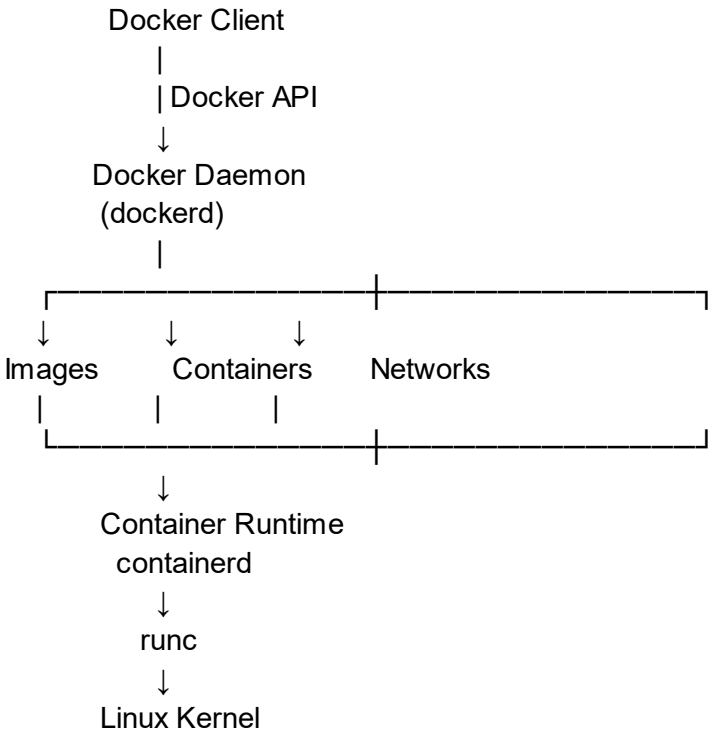
Application
+
Dependencies
+
Libraries
+
Runtime
+
Configuration
↓
Docker Image
↓
Container

So the same image can be deployed across environments.

2. Docker Architecture

Docker follows a **client-server architecture**.

The major components are:



Let's understand each component.

3. Docker Client

The **Docker Client** is what we interact with.

Most commonly, we use the Docker CLI:

docker ps
docker images
docker build
docker run
docker pull
docker push
docker stop

For example:

docker run nginx

The Docker CLI sends the request to the Docker daemon through the Docker API.

Interview answer

> **“Docker Client is the command-line interface through which users interact with Docker. It sends Docker commands to the Docker daemon using the Docker API.”**

4. Docker Daemon — `dockerd`

The **Docker daemon** is the main background service of Docker.

It listens for Docker API requests and manages:

- Containers
- Images
- Networks
- Volumes

For example:

docker run nginx

The flow is:

Docker CLI
|
| docker run nginx
↓
Docker Daemon
|
↓
Check image
|
↓
Create container
|
↓
Start container

Interview answer

> **“Docker daemon, or dockerd, is a background service responsible for managing Docker objects such as images, containers, networks, and volumes.”**

5. Docker Images

A **Docker image** is a read-only template used to create containers.

For example:

nginx:latest
postgres:16
ubuntu:24.04

An image contains things such as:

Application
Libraries
Dependencies
Configuration
Filesystem
Runtime requirements

Example:

docker pull nginx

This downloads the Nginx image from a registry.

6. Docker Container

A **container is a running instance of an image**.

Very important interview point:

Image = Blueprint
Container = Running instance of that blueprint

For example:

Nginx Image
|
├── Container 1
├── Container 2
└── Container 3

Command:

docker run -d --name web nginx

Here:

nginx → Image
web → Container name
-d → Detached mode

7. Docker Registry

A **Docker Registry** stores Docker images.

Examples:

- Docker Hub
- Google Artifact Registry
- Amazon ECR
- Azure Container Registry
- Private registries

Your project is actually using **Google Artifact Registry**, so this is a good example for your interview.

Your image looks like:

us-central1-docker.pkg.dev/
project-7e1069a7-5b78-49e8-9b8/
quickstart-docker-repo/
employment-management:38

The basic flow is:

Developer
|
↓
docker build
|
↓
Docker Image
|
↓
docker push
|
↓
Google Artifact Registry

|
↓
Kubernetes
|
↓
Container

8. Docker Runtime

Modern Docker architecture commonly involves:

Docker CLI
↓
dockerd
↓
containerd
↓
runc
↓
Linux Kernel

containerd

`containerd` is responsible for container lifecycle management.

It handles things such as:

- Creating containers
- Starting containers
- Stopping containers
- Managing container images

runc

`runc` is a low-level container runtime.

It actually creates and starts containers according to the OCI runtime specification.

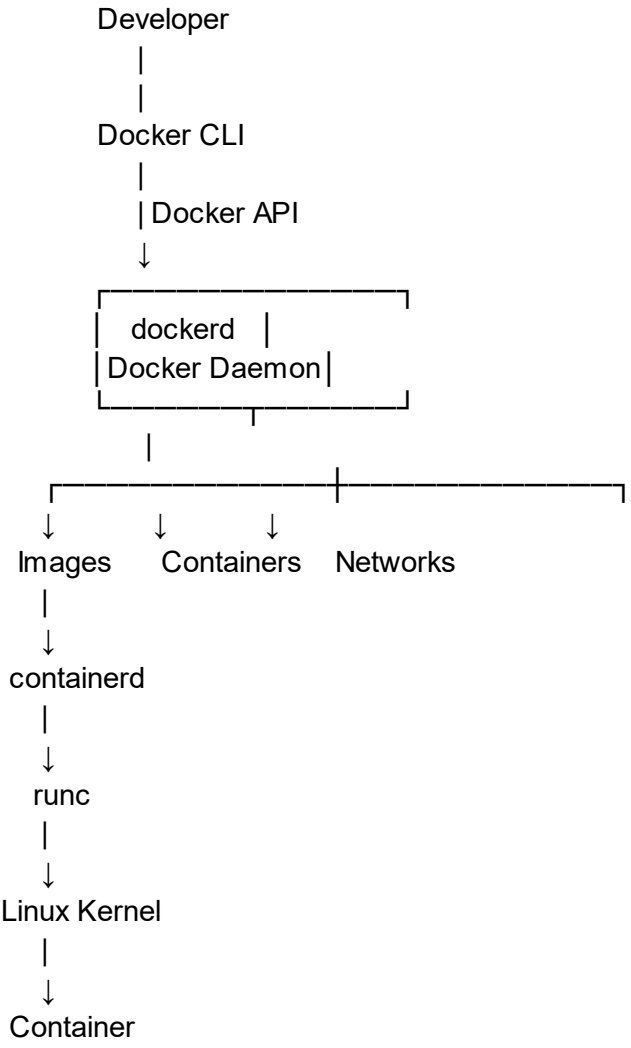
So remember:

dockerd
↓
containerd
↓

runc
↓
Container

9. Docker Architecture Complete Flow

This is the **important diagram to remember for your interview**:



10. Docker Build Process

Suppose you have a Java Spring Boot application.

You create a:

Dockerfile

Example:

```
FROM eclipse-temurin:17-jre

COPY target/employment-management-1.0.0.jar app.jar

EXPOSE 8180

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Then:

```
docker build -t employment-management:1.0 .
```

Flow:

```
Dockerfile
   ↓
docker build
   ↓
Docker Daemon
   ↓
Build image
   ↓
employment-management:1.0
```

Then:

```
docker run -p 8180:8180 employment-management:1.0
```

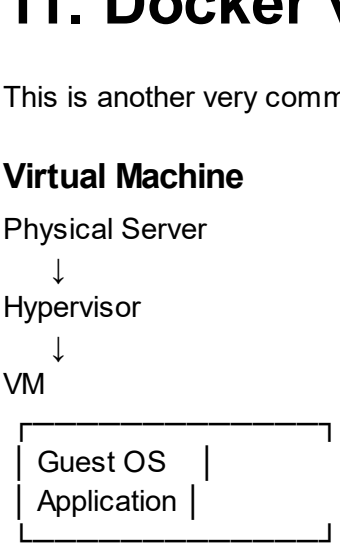
Flow:

```
Docker Image
    ↓
docker run
    ↓
Container
    ↓
Application
    ↓
Port 8180
```

11. Docker vs Virtual Machine

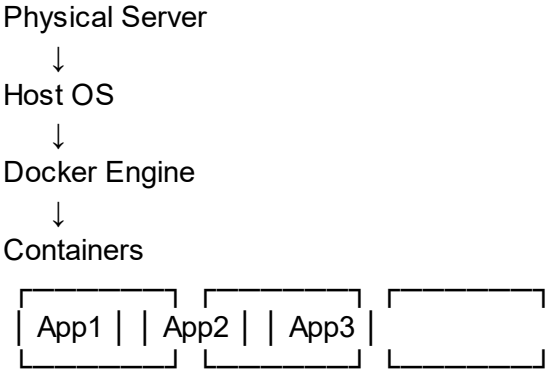
This is another very common interview question.

Virtual Machine



Each VM has its own complete operating system.

Docker



Containers share the host kernel.

Therefore, containers are generally:

- Lightweight
- Faster to start
- More efficient in resource usage
- Easy to package and deploy

12. Real-Time DevOps Example

For your **Employment Management** project, you can explain it like this in an interview:

“In my project, I containerized a Spring Boot application using Docker. First, we package the application as a JAR using Maven. Then we use a Dockerfile to create a Docker image. The image is tagged with the application version and pushed to Google Artifact Registry. Kubernetes then pulls that image from the registry and creates containers from it. This allows us to use the same application image across different environments.”

The complete CI/CD flow:

Developer
↓
GitHub
↓
Jenkins
↓
Maven Build
↓
Unit/Integration Tests
↓
Docker Build
↓
Docker Image
↓
Google Artifact Registry
↓
Kubernetes / GKE
↓
Deployment
↓
Pod
↓
Container
↓
Spring Boot Application

basic Docker commands

Command	Simple meaning
`docker version`	**Shows Docker client and server/version information**

`docker info`	Shows detailed information about Docker
`docker pull`	Downloads an image from a Docker registry
`docker create`	Creates a container but does not start it
`docker rename`	Changes the name of a container
`docker start`	Starts an existing stopped container
`docker stop`	Stops a running container
`docker rm`	Removes/deletes a container
`docker rmi`	Removes/deletes a Docker image
`docker run`	Creates and starts a new container

🐳 What is a Dockerfile?

A Dockerfile is a text file that contains instructions for Docker to build a Docker image.

Think of it as a recipe:

Dockerfile

↓
docker build
↓
Docker Image
↓
docker run
↓
Container

For example:
FROM ubuntu:24.04

WORKDIR /app

COPY app.jar /app/

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]

Dockerfile Instructions

1. FROM

`FROM` specifies the base image.

FROM ubuntu:24.04

Meaning:

> **Start building my image using Ubuntu 24.04 as the base.**

For Java:

FROM eclipse-temurin:17-jre

Interview answer

> **“FROM defines the base image for the Docker image.”**

Almost every Dockerfile starts with `FROM`.

---

2. MAINTAINER

Example:

MAINTAINER Manjunath

It was traditionally used to specify the person who maintains the image.

⚠️ **Important: `MAINTAINER` is deprecated.**

Use:

LABEL maintainer="Manjunath"

instead.

Interview answer

> **"MAINTAINER was used to specify the image maintainer, but it is deprecated and LABEL is preferred."**

---

3. WORKDIR

Defines the working directory inside the container.

WORKDIR /app

Now Docker commands such as `COPY`, `RUN`, and `CMD` can operate relative to `/app`.

Example:

```
WORKDIR /app
COPY app.jar .
```

The file goes to:

/app/app.jar

Interview answer

> **"WORKDIR sets the working directory inside the container."**

---

4. RUN

RUN executes a command while building the image.

Example:

```
RUN apt-get update
RUN apt-get install -y curl
```

The command runs during:

docker build

Important ⭐

RUN → Image build time

Interview answer

> **"RUN executes commands during the Docker image build process."**

---

5. ENV

ENV defines environment variables inside the image/container.

Example:

```
ENV APP_PORT=8080
```

You can use it inside the container:

```
echo $APP_PORT
```

Output:

```
8080
```

Interview answer

> **“ENV is used to set environment variables in the Docker image and container.”**

---

6. ARG

ARG defines a build-time variable.

Example:

```
ARG VERSION=17
FROM eclipse-temurin:${VERSION}-jre
```

You can pass it during build:

```
docker build --build-arg VERSION=21 -t myapp .
```

Important difference ⭐

ARG → Build time
ENV → Available in image/container environment

---

7. LABEL

LABEL adds metadata to an image.

Example:

```
LABEL maintainer="Manjunath"
LABEL version="1.0"
```

It doesn't normally affect how the application runs.

Interview answer

> **“LABEL is used to add metadata such as version, maintainer, or description to a Docker image.”**

---

8. COPY

COPY copies files from the build context into the Docker image.

Example:

COPY app.jar /app/app.jar

Meaning:

Local machine
|
| app.jar
↓
Docker Image
|
↓
/app/app.jar

Interview answer

> **"COPY is used to copy files and directories from the build context into the Docker image."**

---

9. ADD

ADD is similar to COPY, but has some additional capabilities.

Example:

ADD app.tar /app/

ADD can automatically extract local tar archives.

COPY vs ADD ⭐

For normal file copying, generally prefer:

COPY

Use `ADD` when its additional behavior is specifically needed.

Interview answer

> **"ADD copies files into the image and also supports additional features such as extracting local tar archives. For simple copying, COPY is generally preferred."**

---

10. CMD

`CMD` specifies the default command to run when a container starts.

Example:

CMD ["java", "-jar", "app.jar"]

When you run:

docker run myapp

Docker executes:

java -jar app.jar

Important ⭐

RUN → During image build
CMD → When container starts

Interview answer

> **"CMD specifies the default command that runs when a container starts."**

---

11. ENTRYPOINT

`ENTRYPOINT` defines the main executable for the container.

Example:

ENTRYPOINT ["java", "-jar", "app.jar"]

When the container starts, this command runs.

CMD vs ENTRYPOINT ⭐⭐⭐

This is a very common interview question.

CMD

CMD ["java", "-jar", "app.jar"]

CMD can easily be overridden by a command supplied to `docker run`.

ENTRYPOINT

ENTRYPOINT ["java", "-jar", "app.jar"]

It is intended to define the container's main executable.

You can also combine them:

ENTRYPOINT ["java", "-jar"]
CMD ["app.jar"]

Result:

java -jar app.jar

Interview answer

> **“ENTRYPOINT defines the main executable of the container, while CMD provides default arguments or a default command that can be overridden.”**

---

12. USER

`USER` specifies which user should run subsequent commands and/or the container process.

Example:

USER appuser

This is important for security because we generally don't want applications running as root unnecessarily.

Interview answer

> **"USER specifies the user under which commands and the container's main process run."**

---

13. EXPOSE

`EXPOSE` **documents the port that the application expects to listen on.**

Example:

EXPOSE 8080

This does not by itself publish the port to your host.

To publish it:

docker run -p 8080:8080 myapp

Meaning:

Host port 8080
↓
Container port 8080

Interview answer ⭐

> **"EXPOSE documents the port on which the containerized application listens. It does not publish the port; port publishing is done using the `-p` option of `docker run`."**

---

14. ONBUILD

ONBUILD defines an instruction that executes when another Dockerfile uses this image as its base image.

Example:

ONBUILD COPY . /app

It is mainly useful for creating reusable base images.

Simple idea:

Base Image
↓
ONBUILD instruction
↓
Another Dockerfile uses base image
↓
ONBUILD instruction executes

---

15. SHELL

SHELL specifies the shell used by shell-form commands such as RUN.

Example on Linux:

SHELL ["/bin/bash", "-c"]

Instead of the default shell, subsequent shell-form instructions can use Bash.

Interview answer

> **“SHELL specifies the default shell used to execute shell-form commands in the Dockerfile.”**

---

16. STOPSIGNAL

STOPSIGNAL specifies which system signal Docker should send to the container when stopping it.

Example:

STOPSIGNAL SIGTERM

Normally, Docker uses `SIGTERM` as the default stop signal.

Simple meaning:

> **It tells Docker which signal to send when stopping the container.**

---

17. HEALTHCHECK

`HEALTHCHECK` tells Docker how to check whether a containerized application is healthy.

Example:

HEALTHCHECK CMD curl --fail http://localhost:8080/health || exit 1

Docker can then show:

healthy
unhealthy
starting

Check with:

docker ps

Interview answer

> **"HEALTHCHECK is used to determine whether the application inside a container is healthy and responding correctly."**

---

18. VOLUME

`VOLUME` defines a location intended for persistent data.

Example:

VOLUME /var/lib/postgresql/data

This is useful for data that should survive container replacement.

Easy Way to Remember

You don't need to memorize all 18 immediately.

For interviews, focus first on these:

FROM → Base image
WORKDIR → Working directory
RUN → Build-time command
COPY → Copy files
ADD → Copy + additional features
ENV → Environment variable
ARG → Build-time variable
EXPOSE → Document container port
CMD → Default command
ENTRYPOINT → Main executable
USER → Run as specific user
VOLUME → Persistent data
HEALTHCHECK → Check application health

Tag	Digest
Human-friendly name/version	SHA256 hash
Example: `myapp:1.0`	Example: `myapp@sha256:abc123...`
Can be changed/moved	Identifies exact image content
Easy to remember	Long and difficult to remember

Common for development/versioning	**Useful for production and security**
Does not guarantee exact image	**Guarantees exact image**

-

Docker Tag	Docker Digest (SHA256)
Human-readable name/version	SHA256 hash of the image
Example: `myapp:1.0`	Example: `myapp@sha256:abc123...`
Easy to remember	Long and difficult to remember
Can be changed/moved	Identifies the exact image
Commonly used for versioning	Commonly used for production/security
Does not guarantee the exact image	Guarantees the exact image content
Good for development and CI/CD	Good for secure and reproducible deployments

19.What is a Docker multi-stage build?

Answer: A Docker multi-stage build uses multiple `FROM` instructions in a single Dockerfile. Each stage can perform a different task, such as compiling code, running tests, or preparing the final application image.

The main benefit is that we can copy only the required application artifacts into the final image, leaving behind build tools and intermediate files.

Stage 1: Build

Source code + Maven + JDK

Compile and package the application

Copy the JAR file

Stage 2: Runtime

JRE + application JAR

Run the application

Why do we use multi-stage builds?

- Reduce the final image size.
- Avoid including compilers, Maven, and other build tools in the production image.
- Reduce unnecessary packages and potential attack surface.
- Keep Dockerfiles organized around build and runtime tasks.
- Make application delivery more efficient.

What is the difference between a single-stage and a multi-stage build?

Single-stage build	Multi-stage build
Usually uses one build stage	Uses two or more stages

Build tools may remain in the final image	Build tools can be excluded from the final image
Can produce larger images	Can produce smaller images
Build and runtime concerns are combined	Build and runtime concerns are separated

Note: A well-optimized single-stage build can also be small; multi-stage builds make this separation easier.

Explain a multi-stage build with a real-time Java application.

Suppose you have a Spring Boot application that generates a JAR file using Maven.

Create a file named `Dockerfile` in your project root.

```
# Stage 1: Build the application

FROM maven:3.9-eclipse-temurin-21 AS builder


WORKDIR /app


COPY pom.xml .

COPY src ./src


RUN mvn clean package -DskipTests
```

Stage 2: Run the application

FROM eclipse-temurin:21-jre

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]

Explanation of each instruction

- `FROM ... AS builder` — creates the build stage and names it `builder`.
- `WORKDIR /app` — sets the working directory.
- `COPY pom.xml .` — copies the Maven project configuration.
- `COPY src ./src` — copies the source code.
- `RUN mvn clean package -DskipTests` — compiles the application and packages it into a JAR without running tests.
- `FROM eclipse-temurin:21-jre` — starts a fresh runtime stage.
- `COPY --from=builder ...` — copies the JAR from the builder stage into the runtime stage.
- `ENTRYPOINT` — starts the Spring Boot application.

Real-time DevOps scenario

Interviewer: Your Java application image is 1.2 GB. How would you reduce its size?

Sample answer:

“I would inspect the current Dockerfile and identify unnecessary build tools, caches, and files. I would use a multi-stage build to compile the Java application in a Maven/JDK image, then copy only the executable JAR into a suitable JRE runtime image. I would also review the base image, Docker build context, and dependency layers, then compare the resulting image sizes and run tests to confirm that the application works correctly.”

8. Quick revision

Docker instruction	Purpose
`FROM`	Selects a base image and begins a stage
`AS builder`	Names a build stage
`RUN`	Executes a command during the image build
`COPY --from`	Copies artifacts from another stage
`--target`	Selects a stage to build
`ENTRYPOINT`	Defines the main container executable

What is `STOPSIGNAL` in Docker?

`STOPSIGNAL` specifies which operating-system signal Docker sends to a container's main process when the container is stopped.

The default signal is usually `SIGTERM`.

For example:

FROM ubuntu:24.04

STOPSIGNAL SIGTERM

CMD ["sleep", "3600"]

When you run:

docker stop mycontainer

Docker sends `SIGTERM` to the main process. If the process doesn't stop within Docker's timeout, Docker sends `SIGKILL` to force termination.

Common signals:

Signal	Meaning
`SIGTERM`	Requests graceful shutdown
`SIGINT`	Interrupt signal
`SIGQUIT`	Requests termination; behavior depends on the application
`SIGKILL`	Forces immediate termination; cannot be caught or ignored

Important: `STOPSIGNAL` doesn't create a multistage build. It controls shutdown behavior. You can use it in either a single-stage or multistage Dockerfile.

How does `STOPSIGNAL` work in a multistage Dockerfile?

A multistage Dockerfile uses multiple `FROM` instructions. You typically build an application in one stage and copy the compiled artifact into a smaller runtime image in the final stage.

Stage 1: Build

Compile application using Maven or another build tool.

Copy the compiled artifact

Stage 2: Runtime

Run the application with a minimal runtime image and configure `STOPSIGNAL`.

Example using a Java application:

```
# Stage 1: Build the application
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Run the application
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar

STOPSIGNAL SIGTERM
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Explanation

- `FROM ... AS build`: Creates the build stage.

- `COPY --from=build`: Copies the generated JAR from the build stage.
- `FROM eclipse-temurin:21-jre`: Starts a separate runtime stage.
- `STOPSIGNAL SIGTERM`: Configures the stop signal for the final image.
- `ENTRYPOINT`: Starts the Java application.

The final stage's image configuration determines the container's stop signal. Setting `STOPSIGNAL` in the build stage alone does not configure the final stage's signal.

Scenario-based interview questions and answers

Q1. Your application takes time to save data before shutting down. How do you handle this?

Scenario: A Java application is running inside a Docker container. When you stop the container, it exits before completing pending work.

Answer:

Use `STOPSIGNAL SIGTERM` and ensure the application handles `SIGTERM` gracefully. Configure a suitable Docker stop timeout.

STOPSIGNAL SIGTERM

ENTRYPOINT ["java", "-jar", "app.jar"]

Run:

docker stop --timeout=30 myapp

Docker gives the application up to 30 seconds to shut down before forcing termination.

Interview point: The signal requests graceful shutdown; the application must implement the shutdown behavior.

Q2. You set `STOPSIGNAL SIGTERM`, but your container still gets killed. Why?

Possible reasons:

- The application doesn't handle `SIGTERM` correctly.
- The application takes longer than the configured stop timeout.
- The main process is not forwarding signals to the application.
- The application is stuck or cannot finish its cleanup.

Solution: Check container logs, verify the main process, handle shutdown signals properly, and increase the timeout if necessary.

docker logs myapp

docker stop --timeout=60 myapp

Q3. You configured `STOPSIGNAL SIGTERM` in Stage 1. Will it apply to the final image?

Answer: No, not automatically.

FROM maven:3.9-eclipse-temurin-21 AS build

STOPSIGNAL SIGINT

FROM eclipse-temurin:21-jre

STOPSIGNAL SIGTERM

COPY --from=build /app/app.jar /app/app.jar

The final stage uses `SIGTERM`, because its own image configuration determines the stop signal.

Q4. Your container runs Nginx. You want to customize its shutdown signal. What do you do?

Answer: Configure the signal appropriate for the application and verify that Nginx handles it as expected.

FROM nginx:alpine

STOPSIGNAL SIGQUIT

For Nginx, `SIGQUIT` requests a graceful shutdown, while `SIGTERM` requests termination. Test the behavior with your particular image and configuration.

Q5. Docker sends `SIGTERM`, but the container does not stop. What happens next?

Answer: If the stop timeout expires, Docker sends `SIGKILL`, which forcibly terminates the process.

docker stop --timeout=10 myapp

To inspect the result:

docker inspect --format='{{.Config.StopSignal}}' myapp

docker inspect --format='{{.State.ExitCode}}' myapp

The first command checks the configured stop signal. The exit code alone does not prove why the process exited.

Q6. Your multistage Docker image works, but shutdown is unreliable in Kubernetes. What do you investigate?

Answer:

1. Check the final image's configured stop signal.
2. Confirm the application receives termination signals.
3. Ensure the application is the container's main process, preferably using exec-form `ENTRYPOINT`.
4. Check the application's graceful-shutdown configuration.
5. Verify Kubernetes termination grace-period settings.
6. Check logs for unfinished requests or pending database operations.

Kubernetes generally sends `SIGTERM` when terminating a container, then forcefully terminates it after the grace period if it hasn't exited.

Q7. What is the difference between `STOPSIGNAL` and `CMD`?

Instruction	Purpose
`STOPSIGNAL`	Defines the signal used to request container shutdown
`CMD`	Defines the default command or arguments

`ENTRYPOINT`	Defines the executable to run
`EXPOSE`	Documents the port the application listens on

These instructions have different purposes and cannot replace one another.

Q8. You need a small production image and graceful shutdown. How would you design it?

Answer: Use a multistage build, keep build dependencies out of the final runtime image, use an appropriate runtime base image, configure the shutdown signal, and ensure the application handles shutdown correctly.

```
FROM maven:3.9-eclipse-temurin-21 AS build

WORKDIR /src

COPY . .

RUN mvn clean package -DskipTests


FROM eclipse-temurin:21-jre

WORKDIR /app

COPY --from=build /src/target/*.jar app.jar

STOPSIGNAL SIGTERM

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build and test:

```
docker build -t demo-java-app .

docker run -d --name demo-app -p 8080:8080 demo-java-app

docker inspect --format='{{.Config.StopSignal}}' demo-app

docker stop --timeout=30 demo-app
```

docker logs demo-app

1. What is a Docker health check?

A Docker `HEALTHCHECK` instruction tells Docker how to test whether an application inside a container is working correctly.

For example, your Spring Boot application may have a container that is running, but the application itself might be unable to connect to its database or respond to HTTP requests.

Docker can show whether the container is:

- Starting — the health check has not yet succeeded during startup.
- Healthy — the health check is passing.
- Unhealthy — repeated health checks have failed.

Important distinction: a container can be `running` but `unhealthy`. Running means the main process is still running; healthy means the configured health check is passing.

2. Why do we need a health check?

Imagine you deploy a Java application using Docker.

Container is running

The Java process has started.

Application is not responding

The API may be stuck or the application may not have finished initializing.

Health check detects the problem

Docker marks the container unhealthy after the configured failures.

Health checks help with:

1. Detecting applications that are running but not functioning correctly.
2. Monitoring application readiness or responsiveness.
3. Giving monitoring systems a health status to act on.
4. Helping container platforms and deployment tools make better decisions.

Interview point: A Docker health check does not automatically restart an unhealthy container by itself. Docker records the health status; restart policies normally respond to the container process exiting, not merely to its health becoming `unhealthy`.

3. What is a health check in a multistage Docker build?

A multistage build uses multiple `FROM` instructions, commonly to compile an application in one stage and run it in another.

For a Java application:

- Stage 1 — Build: Maven compiles the source code and generates a JAR.
- Stage 2 — Runtime: A smaller Java runtime image runs the JAR and contains the health check configuration.

The health check should normally be configured in the final runtime stage, because that is the image used to run the application.

4. Practical example: Spring Boot + multistage Dockerfile

Suppose your application runs on port `8080` and exposes this Spring Boot Actuator endpoint:

```
http://localhost:8080/actuator/health
```

A successful response looks like:

```
{
  "status": "UP"
}
```

Here is a Dockerfile that builds the application in one stage and runs it with a health check in the second stage.

```
# Stage 1: Build the application
FROM maven:3.9-eclipse-temurin-21 AS build
```

```
WORKDIR /app

COPY pom.xml .

COPY src ./src


RUN mvn clean package -DskipTests


# Stage 2: Runtime image

FROM eclipse-temurin:21-jre


WORKDIR /app


# Install curl so the health check can make HTTP requests

RUN apt-get update \

    && apt-get install -y --no-install-recommends curl \

    && rm -rf /var/lib/apt/lists/*


COPY --from=build /app/target/*.jar app.jar


EXPOSE 8080


# Check the Spring Boot health endpoint

HEALTHCHECK --interval=30s \

            --timeout=5s \
```

```
    --start-period=60s \
    --retries=3 \
    CMD curl --fail --silent http://localhost:8080/actuator/health || exit 1

ENTRYPOINT ["java", "-jar", "app.jar"]
```

This example assumes the Spring Boot Actuator dependency is included and the health endpoint is enabled. The 60-second start period gives the application time to start.

Explanation of each health check option

Option	Meaning
`--interval=30s`	Run the check every 30 seconds.
`--timeout=5s`	Treat a check taking longer than 5 seconds as failed.
`--start-period=60s`	Allow time for initial application startup.
`--retries=3`	Mark the container unhealthy after three consecutive failures outside the startup grace period.
`CMD`	Command Docker executes for the health check.

The command returns success when `curl` receives a successful HTTP response. `--fail` makes HTTP error responses such as 500 cause a failure.

Important: This checks the endpoint's HTTP response. If you need to verify that the database is available too, configure Spring Boot's health endpoint to include database health and ensure that the health check tests the relevant result.

How to run and inspect it

Build the image

docker build -t student-app:1.0 .

Start the container

docker run -d --name student-app -p 8080:8080 student-app:1.0

Check health status

docker ps

Inspect detailed health information

docker inspect --format='{{json .State.Health}}' student-app

View application logs

docker logs student-app

To test a failure, stop the application inside the container or make its health endpoint fail, then inspect the health status. It may take multiple checks before Docker reports `unhealthy`.

When finished:

docker rm -f student-app

5. Scenario-based interview questions and answers

Q1. Your Docker container is running, but the application is not responding. How do you troubleshoot it?

Scenario: Your Spring Boot container shows `Up`, but users get errors when calling the API.

Answer:

I would check the container status, application logs, and health check.

docker ps

docker logs student-app

docker inspect --format='{{.State.Health.Status}}' student-app

If the health status is `unhealthy`, I would inspect the health check's recent output and verify that the endpoint is accessible from inside the container.

Interview takeaway: `running` does not necessarily mean the application is healthy.

Q2. The container becomes unhealthy during application startup. How do you fix it?

Scenario: The Java application needs 45 seconds to start, but the health check starts failing after 10 seconds.

Answer:

Configure a suitable `--start-period` and check interval.

HEALTHCHECK --start-period=60s \

--interval=30s \

--timeout=5s \

--retries=3 \

CMD curl --fail --silent http://localhost:8080/actuator/health || exit 1

I would also verify that the endpoint is available only when the application is ready.

Interview takeaway: Don't make the startup grace period unnecessarily short. The right values depend on actual application startup time.

Q3. The container becomes unhealthy because the database is down. What would you do?

Scenario: The Java process is running, but PostgreSQL is unavailable and the application cannot process requests.

Answer:

- Check the database connection and application logs.
- Configure an appropriate application health endpoint that reflects database connectivity.
- Verify the database hostname, port, credentials, and network connectivity.
- Decide whether database unavailability should make the application unhealthy.

If the application uses Spring Boot Actuator, its health endpoint can report database health when the required health indicator is configured.

Interview takeaway: A simple TCP check against port 8080 may pass even when the database is unavailable. Choose a health check that matches what you want to measure.

. Your image builds successfully, but the health status is always `unhealthy`. What do you check?

Answer:

I would verify the following:

1. Is the application listening on the expected port?
2. Is the endpoint path correct?
3. Does the final image contain `curl` or the chosen health-check executable?
4. Does the endpoint return a successful HTTP response?
5. Is the application still starting?
6. Does the health-check command have the correct syntax?

Commands:

docker logs student-app

docker inspect student-app

docker exec student-app curl -v http://localhost:8080/actuator/health

The last command works only if `curl` exists in the container. Also, `docker exec` requires the container to be running.

How would you explain the complete health-check lifecycle in an interview?

Answer:

1. Docker starts the container.

2. The application initializes.
3. Docker executes the configured health-check command at the configured intervals.
4. A successful check reports healthy; consecutive failures eventually report unhealthy.
5. Docker exposes the status through commands such as `docker ps` and `docker inspect`.
6. An operator, monitoring system, or orchestrator investigates or responds according to its configuration.

The health check monitors the application; it does not, by itself, guarantee that the application will recover.

1. Dangling Docker images

Q1. What are dangling images in Docker?

Answer: Dangling images are images that are untagged and are not referenced by a tag. They commonly appear when rebuilding an image and an older image layer loses its tag.

They often appear with `<none>` for the repository and tag.

Q2. Why do we remove dangling images?

To reclaim disk space and keep the local image cache tidy. However, dangling does not always mean useless: an image may still be useful for build-cache or recovery purposes.

Q3. How do dangling images work?

When you rebuild an image with the same tag, the tag may point to the newly built image. An older image can become untagged if no other tag references it.

Check images:

docker images
docker images -f dangling=true
docker image ls -f dangling=true

Remove dangling images:

docker image prune

Remove all unused images, not just dangling ones:

docker image prune -a

Be careful: `-a` can remove any image not referenced by a container, including images you intended to keep locally.

Scenario-based interview questions

Q4. Your Linux VM is running out of disk space because of Docker. What do you do?

Answer:

docker system df
docker images
docker image prune
docker container prune

I would first inspect Docker disk usage and remove only resources that are safe to delete. I would not blindly run destructive cleanup commands on a production host.

Q5. Does deleting a dangling image delete a running container?

Answer: Normally, no. A running container uses its image layers, and Docker protects images that are still in use. I would still inspect dependencies before cleanup.

2. Multistage Docker builds with Python

Q1. What is a multistage build?

Answer: A multistage build uses multiple `FROM` instructions in a Dockerfile. One stage prepares or builds the application, and another stage contains the files and dependencies needed to run it.

Q2. Why do we use it?

- Reduce the final image size.
- Keep development tools out of the runtime image.
- Separate build and runtime dependencies.
- Reduce unnecessary components in production.

Q3. How does it work?

Build stage

Python + dependency installation + build files

Runtime stage

Only the required application and dependencies

Example Dockerfile:

```
FROM python:3.12-slim AS build
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
WORKDIR /app

COPY --from=build /install /usr/local
COPY app.py .
CMD ["python", "app.py"]
```

Build and run:

```
docker build -t python-app .
docker run --rm python-app
```

This example assumes that `requirements.txt` and `app.py` are in the build context. Native Python dependencies may require compatible build and runtime libraries.

Scenario-based interview questions

Q4. Your Python image is 1 GB. How would you optimize it?

Answer: I would inspect the image layers, use an appropriate slim base image, avoid unnecessary packages and caches, and use a multistage build where useful. Then I would test the application to make sure no runtime dependency was removed.

Q5. What does `COPY --from=build` do?

Answer: It copies selected files from the build stage into the runtime stage. It does not automatically copy the entire build stage.

Q6. Is a multistage build always smaller?

Answer: No. The final size depends on the base image, application files, dependencies, and copied artifacts. I would measure the result with `docker images` and inspect layers with `docker history`.

3. Docker Buildx

Q1. What is Docker Buildx?

Answer: Docker Buildx is a Docker CLI plugin that provides advanced image-building features using BuildKit.

Q2. Why do we use Buildx?

- Build images for different CPU architectures.
- Use advanced build caching.
- Support build secrets and other BuildKit features.
- Build and publish multi-platform images.

Q3. How does it work?

Buildx sends a build definition and context to a BuildKit builder, which executes the build and produces the requested image or images.

Useful commands:

```
docker buildx version
docker buildx ls
docker buildx inspect
docker buildx build -t myapp:1.0 .
```

Example: build for AMD64 and ARM64 and publish to a registry:

```
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t username/myapp:1.0 \
  --push .
```

The builder must support both platforms, and you need permission to push to the registry.

Scenario-based interview questions

Q4. Your organization has both Intel and ARM servers. How do you create a compatible image?

Answer: I would use Buildx with `--platform linux/amd64,linux/arm64`, confirm the application's dependencies support both architectures, and publish the image to a registry.

Q5. Your Buildx command succeeds, but `docker images` does not show the image. Why?

Answer: Depending on the builder and output options, the image may have been built in a separate BuildKit cache or pushed to a registry instead of being loaded into the local Docker image store. For a single-platform local build, I can use:

docker buildx build --load -t myapp:1.0 .

4. Docker build — new ways

Q1. What is BuildKit?

Answer: BuildKit is Docker's modern build engine. It supports efficient caching, parallel execution where possible, and advanced features such as secret mounts.

Difference: BuildKit is the build engine; Buildx is the CLI interface that lets you use many of its features.

Q2. Why do we use BuildKit?

To make builds more efficient, improve cache reuse, and handle advanced build requirements more safely.

Q3. How do we build an image?

docker build -t myapp:1.0 .
docker build --no-cache -t myapp:1.0 .
docker build --pull -t myapp:1.0 .

- Normal build: can reuse cached layers.
- `--no-cache`: does not reuse cached build layers.
- `--pull`: checks for a newer version of the referenced base image.

Q4. What is a Docker build context?

Answer: The build context is the set of files made available to the builder. In `docker build -t myapp .`, the current directory is the context.

Use `.dockerignore` to exclude unnecessary files:

.git
node_modules
.venv
__pycache__

```
*.log
.env
```

Do not copy secrets into images or commit them to source control. Excluding `.env` from the build context helps, but it is not a replacement for proper secrets management.

Scenario-based interview questions

Q5. Every small code change causes your Docker build to take 10 minutes. What would you do?

Answer: I would improve the order of Dockerfile instructions to maximize cache reuse. For example, copy dependency manifests first, install dependencies, and copy frequently changing source code afterward.

Q6. How do you pass a build secret without embedding it in the image?

Answer: Use a BuildKit secret mount. For example:

```
# syntax=docker/dockerfile:1
FROM alpine
RUN --mount=type=secret,id=token \
    test -s /run/secrets/token
```

Build it with:

```
docker build --secret id=token,src=token.txt -t secret-demo .
```

This demonstrates mounting a secret during a build step. In a real application, the build step should consume the secret directly without printing or copying it into an image layer.

5. Distroless images

Q1. What is a Distroless image?

Answer: A Distroless image contains an application runtime and required dependencies without a typical shell, package manager, or general-purpose operating-system utilities.

Q2. Why do we use Distroless?

- Reduce unnecessary software in production.
- Reduce the potential attack surface.
- Avoid shipping build tools and troubleshooting utilities with the application.

Q3. How does it work?

The application is built in a builder image, then only the required application artifact is copied into a suitable Distroless runtime image.

Example for Java:

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM gcr.io/distroless/java21-debian13:nonroot
WORKDIR /app
COPY --from=build /app/target/*.jar /app/app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Verify the selected image tag and Java compatibility before using it.

Scenario-based interview questions

Q4. `docker exec -it myapp sh` fails. What is the reason?

Answer: The Distroless image generally does not include a shell. I would troubleshoot through logs, inspection, metrics, and a compatible debug image.

```
docker logs myapp
docker inspect myapp
docker stats myapp
```

Q5. Does Distroless eliminate all security vulnerabilities?

Answer: No. Application code, runtime components, and dependencies can still contain vulnerabilities. Scanning and patching remain necessary.

6. Scratch vs Distroless

Q1. What is `scratch`?

Answer: `scratch` is an empty starting point for a Docker image. It contains no preinstalled shell, libraries, or utilities.

Q2. What is the difference?

Feature	Scratch	Distroless
Initial contents	Empty	Runtime and required components
Shell	No	Generally no
Package manager	No	No general-purpose package manager
Common use	Self-contained binaries	Applications needing supported runtimes
Setup effort	Often higher	Often easier for supported languages

Scenario-based interview questions

Q3. Which would you choose for a Java Spring Boot application?

Answer: Normally a compatible Distroless Java image because the application requires a Java runtime. With Scratch, I would have to provide all required runtime components myself.

Q4. Which would you choose for a statically linked Go binary?

Answer: Scratch can be suitable if the binary is self-contained and any required certificates or other runtime files are included.

Q5. Can you troubleshoot a Scratch container using a shell?

Answer: Usually not, because Scratch contains no shell. I would use application logs, container inspection, external monitoring, or a separate debugging environment.

7. Image scanning tools

Q1. What is Docker image scanning?

Answer: Image scanning checks a container image for known vulnerabilities in operating-system packages, application dependencies, and sometimes secrets or configuration issues.

Common tools:

- Trivy
- Docker Scout
- Grype

Q2. Why do we use image scanning?

To identify known security risks before deploying an image to production and to support security checks in CI/CD pipelines.

Q3. How does image scanning work?

The scanner examines image layers and package metadata, then compares detected components against vulnerability information.

Example:

trivy image nginx:latest

Scenario-based interview questions

Q4. Your Jenkins pipeline detects a critical vulnerability. What will you do?

Answer:

1. Identify the affected package and vulnerability.
2. Check for a patched version.
3. Update the base image or dependency.
4. Rebuild the image.
5. Scan again.
6. Follow the organization's release policy before deploying.

Q5. The scan reports no vulnerabilities. Is the image completely secure?

Answer: No. Scanners cannot detect every possible flaw. I would also follow secure configuration practices, manage secrets properly, update dependencies, and monitor the running application.

Q6. How can you include scanning in CI/CD?

Answer: Add an image-scanning stage after building the image and before publishing or deploying it. Configure the pipeline to fail on findings that violate the organization's severity policy.

8. Docker container storage

Q1. How does Docker store container data?

Answer: Docker uses image layers and container writable layers. It also supports volumes and bind mounts for persistent data and sharing files with the host.

Q2. What is the difference between a volume and a bind mount?

Docker volume	Bind mount
Managed by Docker	Uses a specific host path
Good for persistent application data	Good for source code and host files
More portable between host directory layouts	Depends on the host path

Q3. How do you create a volume?

```
docker volume create postgres-data
docker volume ls
docker volume inspect postgres-data
```

Example using PostgreSQL:

```
docker run -d \
  --name postgres-db \
  -e POSTGRES_PASSWORD='ChangeMe123!' \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:16
```

This is a lab example, not a recommended production secret configuration.

Scenario-based interview questions

Q4. You deleted a database container. Will you lose the database?

Answer: If the data was stored in an intact named volume, it normally survives container removal. If it existed only in the container writable layer, it may be lost.

Q5. How do you identify what is consuming Docker disk space?

docker system df
docker system df -v
docker images
docker volume ls

I would inspect the usage before deleting resources. I would also verify backups before cleaning up persistent data.

Q6. What happens when you run `docker compose down -v`?

Answer: Compose removes the containers and networks and can remove Compose-managed named volumes. I would avoid using `-v` when persistent database data must be retained.

9. Docker networking

Q1. What is Docker networking?

Answer: Docker networking enables communication between containers, the host, and external systems, subject to the network configuration.

Q2. What are the main network drivers?

Driver	Purpose
`bridge`	Connects containers on one Docker host
`host`	Shares the host network namespace on supported Linux systems
`none`	Disables normal container network connectivity

`overlay`	Supports networking across Swarm nodes

Q3. How do containers communicate?

```
docker network create app-network
docker run -d --name web --network app-network nginx
docker run --rm --network app-network alpine wget -qO- http://web
```

On a user-defined bridge network, Docker provides name resolution for connected containers.

Scenario-based interview questions

Q4. Your backend cannot connect to the PostgreSQL container. What will you check?

Answer:

```
docker ps
docker network ls
docker network inspect app-network
docker logs backend
```

I would verify that both containers share a network, the database hostname and port are correct, PostgreSQL is ready, and credentials are valid.

Q5. Why should a backend not normally use `localhost` to reach a separate database container?

Answer: `localhost` inside the backend container refers to that same container. The backend should use the database service name or another reachable address.

Q6. What is the difference between `EXPOSE 8080` and `-p 8080:8080`?

Answer: `EXPOSE` documents the intended container port. `-p` publishes a host port and maps it to the container port.

10. Docker Compose

Q1. What is Docker Compose?

Answer: Docker Compose is a tool for defining and running multi-container applications using a YAML file, usually named `compose.yaml`.

Q2. Why do we use Docker Compose?

Suppose a project contains a frontend, backend, and PostgreSQL database. Compose lets us define their images, ports, networks, environment variables, volumes, and dependencies in one configuration.

Q3. How does Docker Compose work?

compose.yaml

Services, networks, volumes and configuration

Docker Compose

Builds images and creates resources

Frontend

Backend

Database

Q4. What are the most useful Compose commands?

```
docker compose config
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f
docker compose restart
docker compose down
```

- `config` validates and renders the Compose configuration.
- `build` builds services with build definitions.
- `up -d` starts services in the background.
- `logs -f` follows logs.
- `down` removes the Compose-managed containers and networks.

Scenario-based interview questions

Q5. Your backend starts before PostgreSQL is ready. How do you fix it?

Answer: Add a PostgreSQL health check and use `depends_on` with `condition: service_healthy`. Also implement connection retries in the backend, because readiness at startup does not guarantee future availability.

Q6. What is the difference between `docker compose up` and `docker compose up -d`?

Answer: The first runs in the foreground and displays logs; `-d` runs the services in the background.

Q7. How do Compose services communicate?

Answer: Services on the same Compose network can normally communicate using their service names as hostnames.

11. Docker Swarm

Q1. What is Docker Swarm?

Answer: Docker Swarm is Docker's built-in orchestration system for managing services across multiple Docker hosts.

Q2. Why do we use Docker Swarm?

- Run multiple replicas of an application.
- Schedule containers across cluster nodes.
- Support service discovery and load balancing.
- Perform rolling service updates.
- Reconcile service tasks toward the desired replica count.

Q3. What are manager and worker nodes?

Manager nodes maintain cluster state and schedule tasks. Worker nodes run assigned tasks. Managers can also run application tasks unless configured otherwise.

Q4. How do you create a Swarm service?

Initialize a Swarm on a manager node:

```
docker swarm init
docker node ls
```

Create a service with three replicas:

```
docker service create \
  --name web \
  --replicas 3 \
  --publish 8080:80 \
  nginx
```

Inspect it:

```
docker service ls
docker service ps web
docker service logs web
```

Scenario-based interview questions

Q5. One of three replicas fails. What happens?

Answer: Swarm attempts to restore the desired replica count. I would check task status and logs to identify the underlying failure.

Q6. How do you scale from three replicas to five?

```
docker service scale web=5
docker service ps web
```

Swarm attempts to schedule five replicas, subject to resource availability and scheduling constraints.

Q7. What happens if a worker node goes down?

Answer: The manager detects that the node is unavailable and may schedule replacement tasks on other eligible nodes. Recovery depends on available capacity, placement constraints, and the health of the cluster.

Q8. What is the difference between Docker Compose and Docker Swarm?

Docker Compose	Docker Swarm
Defines multi-container applications	Orchestrates services across a cluster
Commonly used for local development	Provides cluster-level deployment

Uses `docker compose up`	Uses `docker service` commands
Does not itself provide multi-node orchestration	Schedules tasks across Swarm nodes

12. Compose example 1 — Python application with PostgreSQL

Scenario: You are a DevOps engineer. A developer gives you a Python application that needs PostgreSQL. Your task is to start both services with a single command and preserve database data.

Create this directory structure:

```
python-project/
├── app.py
├── requirements.txt
└── compose.yaml
```

`requirements.txt`:

```
flask
psycopg2-binary
```

`app.py`:

```
from flask import Flaskapp = Flask(__name__)@app.route("/")def home():    return "Python application is running!"app.run(host="0.0.0.0", port=5000)
```

`compose.yaml`:

```
services:
  app:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      db:
        condition: service_healthy

  db:
```

```
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres-data:
```

Add a `Dockerfile` for the Python application:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

Create a local `.env` file containing a suitable lab password:

```
DB_PASSWORD=ReplaceWithALabPassword
```

Run the project:

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs -f
```

Open `http://localhost:5000` in your browser.

Note: This basic Flask example demonstrates Compose, health checks, and storage. It does not yet connect the Python application to PostgreSQL; the database connection logic would need to be added separately. Keep `.env` out of Git and use proper secrets management in production.

Scenario-based interview questions

Q1. Why is the database using a named volume?

Answer: To preserve database data when its container is recreated.

Q2. Why does the application listen on `0.0.0.0`?

Answer: So the server accepts connections through the container's network interfaces. Binding only to `127.0.0.1` inside the container would prevent normal external access through the published port.

Q3. Why use `condition: service_healthy`?

Answer: It delays the dependent service's startup until the database health check succeeds. The application should still handle database failures after startup.

13. Compose example 2 — Java Spring Boot with PostgreSQL

Scenario: You need to deploy a Spring Boot application and PostgreSQL using Docker Compose.

Directory structure:

```
spring-project/
├── Dockerfile
├── compose.yaml
├── pom.xml
└── src/
```

Dockerfile using a multistage build:

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Compose file:

```
services:
  backend:
    build: .
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/studentdb
      SPRING_DATASOURCE_USERNAME: student
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: studentdb
      POSTGRES_USER: student
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - student-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U student -d studentdb"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  student-data:
```

Create a `.env` file with `DB_PASSWORD` set to a suitable lab password. Ensure the Spring Boot project includes the PostgreSQL JDBC driver and compatible datasource configuration.

Run:

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs -f backend
docker compose logs -f db
```

Scenario-based interview questions

Q1. Spring Boot fails with a database connection error. What do you check?

Answer: I check the database logs and health status, confirm the hostname is `db` rather than `localhost`, verify the port, credentials, JDBC driver, and datasource configuration, and check whether the application retries failed connections.

Q2. You updated Java source code. How do you rebuild the application?

Answer:

docker compose up -d --build backend

Compose rebuilds the backend image and recreates the service when necessary.

Q3. How do you confirm the application is working?

Answer: Check the container status and logs, then test the application's endpoint. If the application exposes Spring Boot Actuator, its health endpoint can also help verify readiness.

Q4. How do you stop the application without removing the database volume?

docker compose down

Scenario 1: Backend container starts before the database

Interviewer: Your Spring Boot backend starts, but PostgreSQL is not ready. The application fails with a database connection error. How will you fix it?

ಕನ್ನಡದಲ್ಲಿ: Backend container ಮೊದಲು start ಆಗುತ್ತದೆ. ಆದರೆ database ಇನ್ನೂ connections ಸ್ವೀಕರಿಸಲು ಸಿದ್ಧವಾಗಿಲ್ಲ. ಏನು ಮಾಡುತ್ತೀರಿ?

Answer (English): I would configure a PostgreSQL health check and use `depends_on` with `condition: service_healthy`. I would also implement database connection retries in the application.

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: example123
      POSTGRES_DB: appdb
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
```

```
    interval: 10s
    timeout: 5s
    retries: 5

  backend:
    image: my-backend:1.0
    depends_on:
      db:
        condition: service_healthy
```

Why? `depends_on` alone does not necessarily mean the database is ready to accept connections. A health check helps with startup readiness.

Scenario 2: Backend cannot connect to PostgreSQL

Interviewer: Your backend is running, but it shows `Connection refused` or a database connection error. How do you troubleshoot?

Answer (English): I would check the database container status, logs, network connectivity, hostname, port, and credentials.

```
docker compose ps
docker compose logs db
docker compose logs backend
docker network ls
```

Important interview point: Inside the backend container, use the Compose service name as the database hostname, for example:

```
jdbc:postgresql://db:5432/appdb
```

Do not use `localhost` to refer to another container. Inside a container, `localhost` refers to that same container.

Scenario 3: Database data disappears after restarting containers

Interviewer: You restart your Compose application, but your database tables and records are missing. What is the likely reason?

Answer (English): The database data may not have been stored in a persistent volume, or the volume may have been removed or mounted incorrectly. I would verify the volume configuration and inspect the database's data directory.

```
docker volume ls
docker volume inspect postgres-data
docker compose down
docker compose up -d
```

Example:

```
services:
  db:
    image: postgres:16
    volumes:
      - postgres-data:/var/lib/postgresql/data

volumes:
  postgres-data:
```

Remember: `docker compose down` normally retains named volumes. `docker compose down -v` can remove Compose-managed volumes, so use it carefully.

Scenario 4: A developer changes the application code

Interviewer: The developer has updated the Spring Boot code. How do you deploy the changes through Compose?

Answer (English): I would rebuild the backend image and recreate the service.

```
docker compose up -d --build backend
docker compose ps
docker compose logs -f backend
```

Why? Changing source code on the host does not automatically update an already-built image unless a suitable development bind mount or hot-reload setup is configured.

Scenario 5: A container exits immediately

Interviewer: Your container starts and exits after a few seconds. How do you investigate?

Answer (English): I would inspect the container's logs, exit code, command, environment variables, and dependencies.

```
docker compose ps -a
docker compose logs backend
docker inspect <container_name>
```

Possible causes include:

- Application startup failure.
- Incorrect `ENTRYPOINT` or `CMD`.
- Missing environment variables.
- Failed database connections.
- The main process exits normally.

Interview tip: A container normally stays running only while its main process is running.

Scenario 6: One service is consuming too many resources

Interviewer: Your backend container consumes too much memory and affects other services. What will you do?

Answer (English): I would inspect resource usage, review application memory settings, and configure appropriate container resource limits where supported.

```
docker stats
docker compose ps
```

For a Compose deployment that supports these settings, resource limits can be specified through the applicable Compose configuration. I would verify that the limits are actually enforced by the target runtime.

Why? Resource controls can help prevent one service from exhausting the host's resources.

Scenario 7: You need to troubleshoot a Compose application quickly

Interviewer: Your frontend returns an error, but the backend and database appear to be running. What commands do you use?

Answer (English): I would validate the Compose configuration, inspect service status, check logs, and test connectivity between services.

```
docker compose config
docker compose ps
docker compose logs --tail=100
docker network inspect <network_name>
```

Then I would verify the frontend's backend URL, port mappings, DNS resolution, and application configuration.

Scenario 8: The same application must run on different developers' machines

Interviewer: How does Docker Compose help your team?

Answer (English): Compose provides a repeatable service configuration. Team members can use the same YAML file to start the application and its dependencies, reducing configuration differences.

docker compose up -d --build

I would keep environment-specific settings outside the shared configuration and avoid committing credentials to Git.

Part 2: Docker Swarm — Important scenarios

Scenario 9: One replica crashes in production

Interviewer: Your Swarm service has five replicas. One replica crashes. What happens?

Answer (English): Swarm attempts to reconcile the service with its desired state and replace failed tasks when possible. I would inspect task status and logs to determine the root cause.

docker service ls
docker service ps myapp
docker service logs myapp

Why? Swarm continuously attempts to maintain the requested service state, subject to scheduling constraints and available resources.

Important: A container that remains running but is unhealthy is not necessarily restarted automatically just because of its health status.

Scenario 10: You need to scale from three replicas to ten

Interviewer: Traffic has increased. How do you scale the application?

Answer (English): I would increase the desired replica count using Docker Swarm.

```
docker service scale myapp=10
docker service ls
docker service ps myapp
```

Why? Swarm attempts to schedule the additional tasks on eligible nodes.

Follow-up: Does scaling replicas always make the application faster?

Answer: No. The application may be limited by the database, CPU, memory, network, or an external dependency. I would measure the bottleneck before scaling.

Scenario 11: A worker node goes down

Interviewer: One worker node becomes unavailable. How does Swarm respond?

Answer (English): The manager detects the node's unavailable state and may schedule replacement tasks on other eligible nodes, provided sufficient resources are available.

```
docker node ls
docker service ps myapp
```

Why? The cluster attempts to restore the desired service state.

Important: Recovery is not guaranteed if the remaining nodes lack resources or satisfy none of the placement constraints.

Scenario 12: You need to deploy an application across three servers

Interviewer: Your application must run across multiple Docker hosts. Would you choose Compose or Swarm?

Answer (English): If I need native multi-node orchestration using Docker's built-in capabilities, I would consider Docker Swarm. Compose alone does not provide Swarm-style scheduling across multiple nodes.

Initialize a cluster on the manager:

```
docker swarm init
```

On another host, use the join command generated by the manager:

docker swarm join --token <worker-token> <manager-ip>:2377

Check nodes on the manager:

docker node ls

Security note: Treat join tokens as credentials. Use the actual command and token produced by your manager.

Scenario 13: A Swarm service cannot start all replicas

Interviewer: You request ten replicas, but only six are running. What will you check?

Answer (English): I would inspect task errors, node availability, resource capacity, image-pull failures, and placement constraints.

```
docker service ps myapp
docker node ls
docker service inspect myapp
```

Possible causes:

- Not enough CPU or memory.
- Unavailable worker nodes.
- Registry authentication problems.
- Placement constraints that exclude nodes.
- Application startup failures.

Scenario 14: You need to update an application with minimal downtime

Interviewer: How would you update a running Swarm service?

Answer (English): I would use a controlled rolling update, configure the update behavior, and monitor the service during deployment.

Example:

```
docker service update \
  --image nginx:1.28 \
  --update-parallelism 1 \
  --update-delay 10s \
  myapp
```

This example assumes the service is intended to run the specified Nginx image. For a real application, use the correct, tested image tag.

Why? Rolling updates replace tasks in controlled batches instead of attempting to replace every task at once.

Follow-up: What if the new version fails?

Answer: I would inspect task status and logs, then use the service rollback mechanism if appropriate and if the previous service specification is suitable.

docker service rollback myapp

Scenario 15: You need to restrict where a service runs

Interviewer: Your database should run only on nodes labeled for database workloads. What would you do?

Answer (English): I would label the appropriate node and configure a placement constraint for the service.

Example:

docker node update --label-add role=database worker1

For a service created with the constraint:

```
docker service create \
  --name db \
  --constraint 'node.labels.role==database' \
  postgres:16
```

Why? Placement constraints restrict scheduling to nodes that meet the specified conditions.

Important: A placement constraint is not a complete security boundary. I would also configure network isolation, access controls, and secret management.

Scenario 16: A Swarm service is published but cannot be accessed

Interviewer: The service is running, but users cannot connect to it. How do you troubleshoot?

Answer (English): I would verify service status, port publishing, firewall rules, application listening ports, and network configuration.

docker service ls
docker service ps myapp
docker service inspect myapp

I would also verify the required Swarm communication ports and ensure the published application port is permitted by the host and cloud firewalls.

Why? A running task does not guarantee that external network access is correctly configured.

Part 3: Advanced Compose vs Swarm interview questions

Scenario 17: Can Docker Compose restart a failed container?

Answer (English): Compose can configure container restart policies, such as `restart: unless-stopped`. However, a restart policy primarily responds to container termination; it does not automatically restart a container solely because its health check reports `unhealthy`.

Example:

```
services:
  app:
    image: nginx:alpine
    restart: unless-stopped
```

In Swarm, the orchestrator manages service tasks and attempts to maintain the desired replica count.

Scenario 18: Can Docker Compose provide high availability across multiple hosts?

Answer (English): Standard `docker compose up` is not a multi-node orchestrator. For multi-node scheduling and service reconciliation, I would consider Docker Swarm or another orchestration platform.

Scenario 19: What happens if the Swarm manager fails?

Answer (English): If other managers remain and a majority quorum is available, the cluster can continue manager operations. If a single-manager cluster loses its manager, existing tasks may continue running, but cluster management and scheduling are impaired until management is restored.

Interview tip: For production, use an appropriate odd number of manager nodes—commonly three or five—to improve quorum resilience.

Scenario 20: How do you securely manage passwords in Compose and Swarm?

Answer (English): I would avoid hardcoding passwords in Dockerfiles, source control, or committed Compose files. I would use an approved secrets manager or supported secrets mechanisms, limit access, rotate credentials, and avoid exposing secrets in logs.

In Swarm, Docker secrets can be used with supported services. For example, a service can be granted access to a named secret, and the application can read the mounted secret file.

Part 4: Quick interview challenge

Try answering these before opening the suggested answers.

1. Compose starts the database container, but the backend fails immediately.
2. A Swarm service needs 8 replicas, but only 5 are running.
3. You want the database data to survive container recreation.
4. Your service update introduces a serious error.
5. A backend container uses localhost to connect to PostgreSQL in another container.
6. A container is unhealthy but has not restarted.
7. A Swarm worker goes offline.
8. You need one command to start a local frontend, backend and database.

What you should remember for the interview

- Compose: Defines and runs a multi-container application.
- Swarm: Orchestrates services across multiple Docker nodes.
- Compose health checks: Help determine whether a service is ready.
- Swarm replicas: Express the desired number of service tasks.
- Persistent volumes: Protect data from ordinary container recreation.
- Troubleshooting: Start with status, logs, inspection, networking, and resource usage.